SIMATS ENGINEERING

Department of Artificial Intelligence and Data Science

DSA04 – FUNDAMENTALS OF DATA SCIENCE

ASSIGNMENT 2 – CO2

STUDENT PERFORMANCE DATA PREPARATION AND VISUALIZATION USING PYTHON

Student Details	Information
Student Name	P V GIRIDHAR
Register Number	1972260022
Department	Artificial Intelligence and Data Science
Academic Year	2025 – 2029
Faculty Name	Dr Vanaja Ramalingam
Assignment Type	Individual
Duration	1 Week
Total Marks	50
Date of Submission	__ / __ / 2026
Date of Evaluation	__ / __ / 2026

Mark Distribution

Task	Title	Maximum Marks
1	Inspecting and Reading the Dataset	4
2	Data Selection and Filtering	4
3	Identifying and Handling Missing Data	5
4	Sorting, Grouping and Ranking	5
5	Data Manipulation Using Pandas and NumPy	5
6	Data Preprocessing Workflow	4
7	Data Visualization Using Matplotlib	6
8	Choosing Suitable Python Libraries	4
9	Interpreting Visualized Data for Academic Support	6
10	Integrated Preprocessing and Visualization Case Study	7
	TOTAL	50

INTRODUCTION AND REPORT SCOPE

An engineering college collects student information from its student information system, attendance records, internal assessments and learning management system (LMS). Raw data from several systems is rarely ready for analysis: it may contain missing entries, repeated records, inconsistent spellings and impossible values. This assignment demonstrates how such a dataset can be inspected, cleaned, manipulated, summarised and visualised using Python libraries (Pandas, NumPy, SciPy, Scikit-learn and Matplotlib) so that the results can be used to identify students who may need academic support.

The report follows the ten tasks in the assignment brief. Every task contains the Python code, the place where the output screenshot is pasted, and an interpretation written from the results.

Dataset used

A small, clearly identified **sample dataset of 22 raw records (20 unique students)** was created for this assignment using the program 00_create_dataset.py and saved as students_raw.csv. It is a hypothetical dataset prepared for learning. Student names are coded (Student_01 to Student_20), so no personally identifiable information is used. Data-quality problems were deliberately included so that every preprocessing technique can be demonstrated.

Column	Meaning	Type expected
Student_ID	Coded student identifier (S101 – S120)	Text
Student_Name	Coded name (Student_01 – Student_20)	Text
Department	CSE, ECE, MECH or AIDS (entered with inconsistent spellings)	Text (category)
Attendance	Attendance percentage	Number
Internal_Marks	Internal assessment marks (out of 100)	Number
Assignment_Score	Assignment marks (out of 20)	Number
Online_Hours	Weekly online-learning (LMS) hours	Number

Data-quality problem included in the raw file	Where it occurs
Duplicate records	S103 and S110 each appear twice (22 rows, 20 students)
Missing values	Attendance: 2 (S106, S119); Internal marks: 1 (S111); Assignment score: 3 (S104, S117, S120)
Inconsistent department names	11 spellings for 4 departments, e.g. CSE, cse, C.S.E, Computer Science, Ece, ECE with a trailing space, mech, Mechanical
Incorrect data type	Attendance written as text with a percent sign in three records (92%, 95%, 90%)
Invalid value	Attendance of 105% for S115 (cannot exceed 100)
Unusually low mark	S118 has internal marks of 12 while the class median is 66

Working assumptions

- Identical Student_ID rows are duplicates; the first occurrence is kept.
- Valid ranges are: attendance 0–100, internal marks 0–100, assignment score 0–20. Values outside these ranges are treated as missing.
- Missing values are filled with the median unless a task states otherwise. Unusual marks are flagged using the IQR rule but not deleted, because they may be genuine.
- The 75% attendance limit and the marks thresholds used in the report are chosen for illustration and are stated where used.
- All records and results are hypothetical and are not real institutional data.

Program files and order of execution

Place all program files in one folder, open each file in IDLE (File → Open) and press **F5** to run it. The raw dataset must be created first, and task6_preprocessing.py must be run before Tasks 4, 5, 7 and 9 because it produces the cleaned file students_clean.csv used by those programs.

Order	Program file	Purpose	Needs
1	00_create_dataset.py	Creates students_raw.csv (run once)	–
2	task1_inspect.py	Task 1 – reading and inspection	students_raw.csv
3	task2_selection.py	Task 2 – selection and filtering	students_raw.csv
4	task3_missing.py	Task 3 – missing-data handling	students_raw.csv
5	task6_preprocessing.py	Task 6 – full cleaning workflow; saves students_clean.csv	students_raw.csv
6	task4_sort_group_rank.py	Task 4 – sorting, grouping, ranking	students_clean.csv
7	task5_manipulation.py	Task 5 – Pandas / NumPy manipulation	students_clean.csv
8	task7_visualization.py	Task 7 – three Matplotlib plots	students_clean.csv
9	task8_libraries_demo.py	Task 8 – demonstration of the five libraries	students_clean.csv
10	task9_interpretation.py	Task 9 – patterns and support list	students_clean.csv
11	task10_case_study.py	Task 10 – integrated workflow from raw data	students_raw.csv

Libraries required: pandas, numpy, matplotlib, scipy and scikit-learn (install with: pip install pandas numpy matplotlib scipy scikit-learn). In newer Pandas versions text columns are displayed with the type name str instead of object; both mean text.

TASK 1 – Inspecting and Reading the Dataset (4 Marks)

Demonstrate how the student performance dataset can be loaded and examined before preprocessing.

1.1 Python code

Python code *(file: task1_inspect.py)*

```
# TASK 1 - Inspecting and Reading the Dataset
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_raw.csv")

print("===== FIRST 5 RECORDS =====")
print(df.head())

print("\n===== LAST 5 RECORDS =====")
print(df.tail())

print("\n===== DATASET DIMENSIONS (rows, columns) =====")
print(df.shape)

print("\n===== COLUMN NAMES =====")
print(df.columns.tolist())

print("\n===== DATA TYPES =====")
print(df.dtypes)

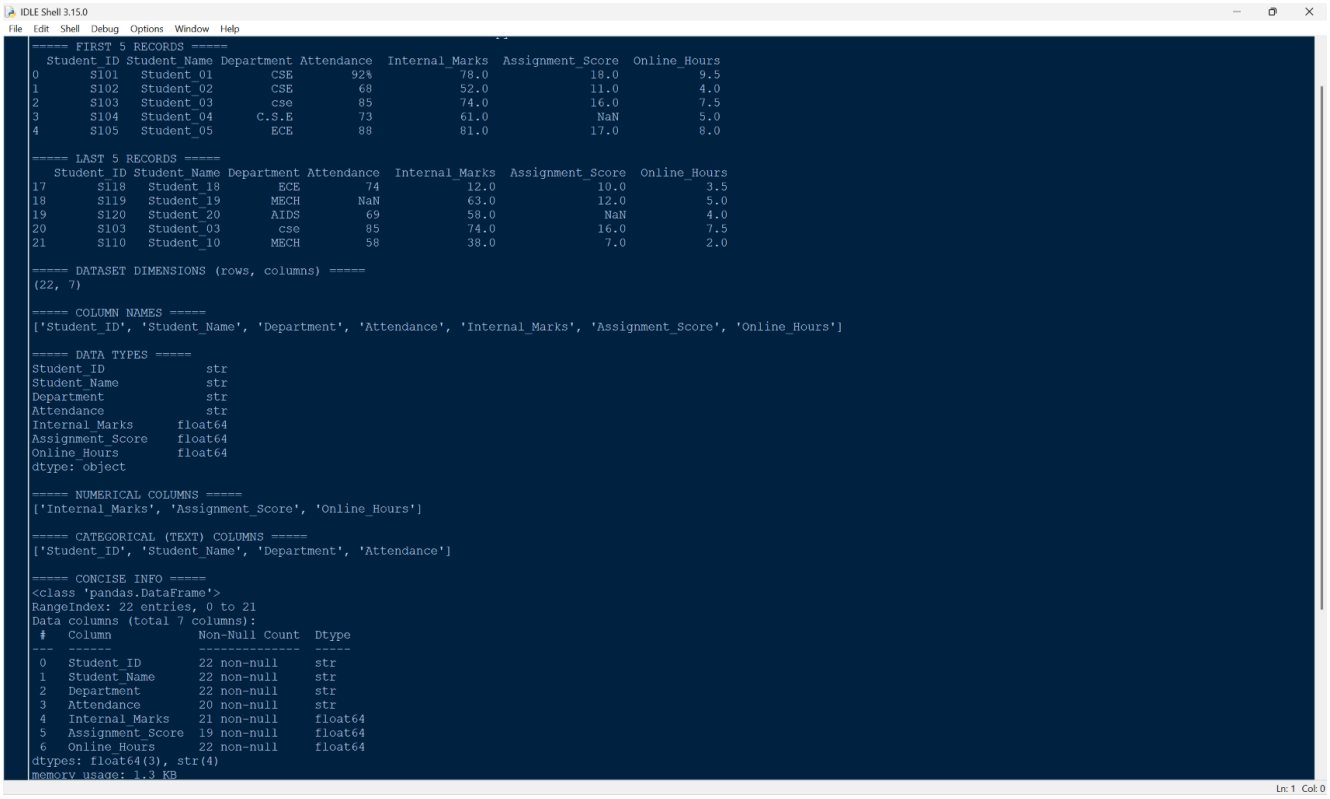
print("\n===== NUMERICAL COLUMNS =====")
print(df.select_dtypes(include="number").columns.tolist())

print("\n===== CATEGORICAL (TEXT) COLUMNS =====")
print(df.select_dtypes(exclude="number").columns.tolist())

print("\n===== CONCISE INFO =====")
df.info()

print("\n===== QUICK DATA-QUALITY INDICATORS =====")
print("Missing values per column:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
print("Unique department labels:", df["Department"].unique())
```

Output – Screenshot 1



1.2 Interpretation of the initial inspection

The CSV file was read into a DataFrame with **pd.read_csv()**. The head() and tail() commands show the first and last five records, shape shows the size, columns lists the column names, dtypes gives the data type of each column and info() shows how many non-null values every column holds.

Observation	Result from the inspection
Dimensions	22 rows × 7 columns (22 raw records for 20 unique students)
Column names	Student_ID, Student_Name, Department, Attendance, Internal_Marks, Assignment_Score, Online_Hours
Numerical columns	Internal_Marks, Assignment_Score, Online_Hours (float64)
Categorical / text columns	Student_ID, Student_Name, Department and Attendance (stored as text)
Missing values	Attendance 2, Internal_Marks 1, Assignment_Score 3 (info() shows 20, 21 and 19 non-null values out of 22)
Duplicate rows	2 (the last two rows repeat S103 and S110)

Two data-quality concerns identified:

- **Wrong data type and missing entries in Attendance.** Attendance is expected to be numeric, but Pandas read it as text because some entries contain a percent sign (for example 92%). Arithmetic and comparisons such as “attendance below 75” cannot be done reliably until it is converted. The column also has missing entries, as do the marks and assignment columns.

- **Inconsistent department labels and duplicate rows.** unique() shows 11 different department spellings for only four departments, so a grouping would split one department into several groups. In addition, two students appear twice, which would distort averages and counts.

TASK 2 – Data Selection and Filtering (4 Marks)

Select relevant information from the student dataset to support academic performance analysis.

2.1 Python code

Attendance is first converted to numbers (the percent sign is removed) so that it can be compared with 75. The chosen internal-marks threshold is **70**.

Python code *(file: task2_selection.py)*

```
# TASK 2 - Data Selection and Filtering
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_raw.csv")

# Attendance contains text such as "92%", so convert it to numbers first
df["Attendance"] = pd.to_numeric(df["Attendance"].astype(str).str.replace("%", ""),
errors="coerce")

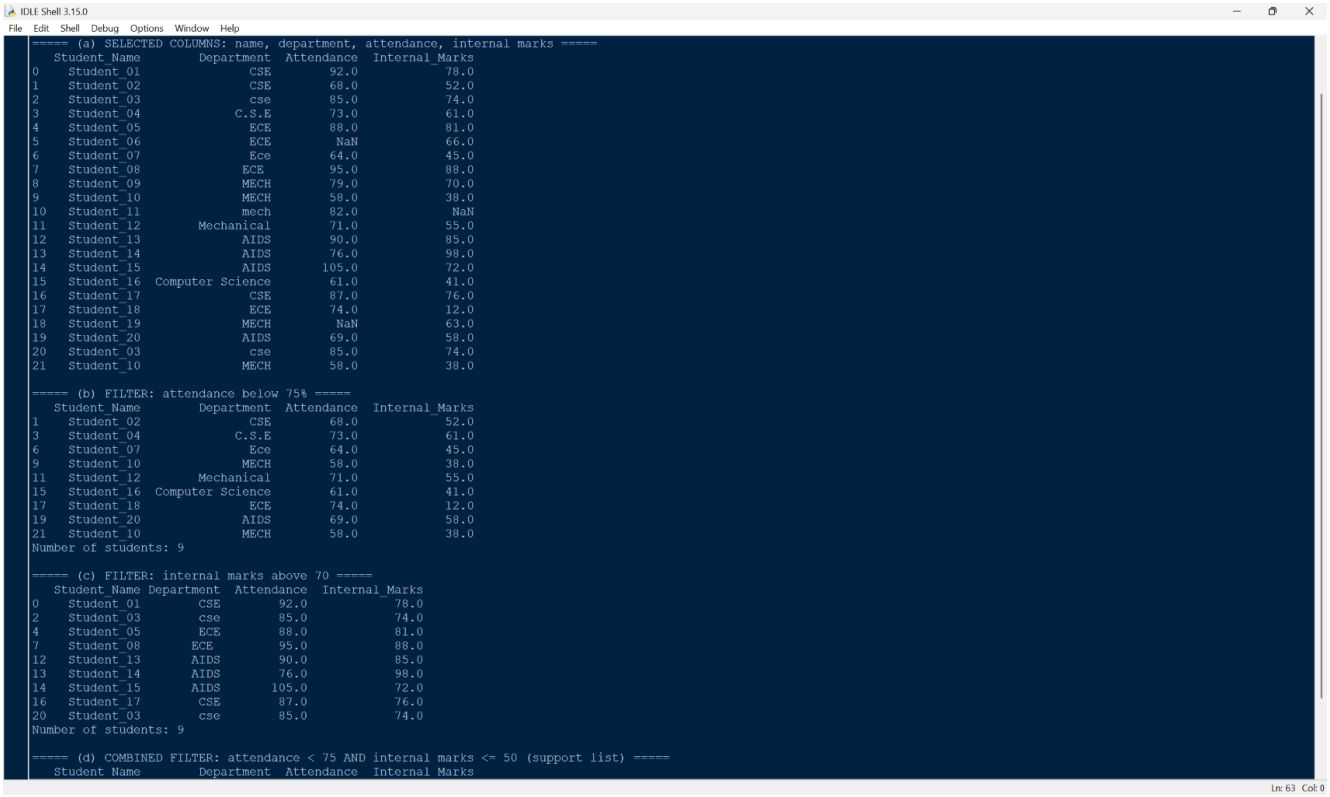
print("===== (a) SELECTED COLUMNS: name, department, attendance, internal marks =====")
selected = df[["Student_Name", "Department", "Attendance", "Internal_Marks"]]
print(selected)

print("\n===== (b) FILTER: attendance below 75% =====")
low_att = selected[selected["Attendance"] < 75]
print(low_att)
print("Number of students:", len(low_att))

THRESHOLD = 70      # chosen threshold for internal marks
print(f"\n===== (c) FILTER: internal marks above {THRESHOLD} =====")
high_marks = selected[selected["Internal_Marks"] > THRESHOLD]
print(high_marks)
print("Number of students:", len(high_marks))

print("\n===== (d) COMBINED FILTER: attendance < 75 AND internal marks <= 50 (support
list) =====")
support = selected[(selected["Attendance"] < 75) & (selected["Internal_Marks"] <= 50)]
print(support)
```

Output – Screenshot 2



2.2 Interpretation

Selection / filter	Pandas statement	Result
Selected columns	df[[“Student_Name”, “Department”, “Attendance”, “Internal_Marks”]]	4 columns, 22 rows
Attendance below 75%	selected[selected[“Attendance”] < 75]	9 rows = 8 students (Student_10 appears twice because of the duplicate record)
Internal marks above 70	selected[selected[“Internal_Marks”] > 70]	9 rows = 8 students (Student_03 appears twice)
Low attendance AND marks ≤ 50	(Attendance < 75) & (Internal_Marks <= 50)	5 rows = 4 students: Student_07, Student_10, Student_16, Student_18

How the filtered records help identify students who need support. The attendance filter gives an early-warning list of eight students. Attendance alone is not enough, however: Student_04, Student_12 and Student_20 have low attendance but still obtained 55–61 marks, so a reminder or monitoring is probably enough for them. The combined filter isolates four students (Student_07, Student_10, Student_16 and Student_18) who have low attendance **and** very low marks. These are the most suitable candidates for mentoring or remedial classes. The high-marks filter shows the opposite group, who can act as peer tutors.

Points noticed while filtering. (1) The two duplicate records inflate the counts, so duplicates must be removed before any report is prepared. (2) Students whose attendance is missing (Student_06 and Student_19) do not appear in the “below 75” list at all, because a comparison with a missing value is false; missing data can therefore hide students. (3) Student_15 appears in the high-marks list with 105% attendance, an impossible value that is corrected in Task 6.

TASK 3 – Identifying and Handling Missing Data (5 Marks)

The dataset contains missing values in attendance, internal marks and assignment scores. Develop a suitable strategy to handle them.

3.1 Python code

Duplicates are removed first so that all counts refer to the 20 unique students. Two methods are demonstrated: **Method 1 – deletion** (dropna) and **Method 2 – replacement** (fillna with median or mean).

Python code *(file: task3_missing.py)*

```
# TASK 3 – Identifying and Handling Missing Data
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_raw.csv")
df["Attendance"] = pd.to_numeric(df["Attendance"].astype(str).str.replace("%", ""),
errors="coerce")
df = df.drop_duplicates()                       # duplicates removed so counts are per
student

print("===== MISSING VALUES: COUNT PER COLUMN =====")
print(df.isnull().sum())
print("\n===== MISSING VALUES: PERCENTAGE PER COLUMN =====")
print((df.isnull().mean() * 100).round(1))

print("\n===== ROWS CONTAINING MISSING VALUES =====")
print(df[df.isnull().any(axis=1)])

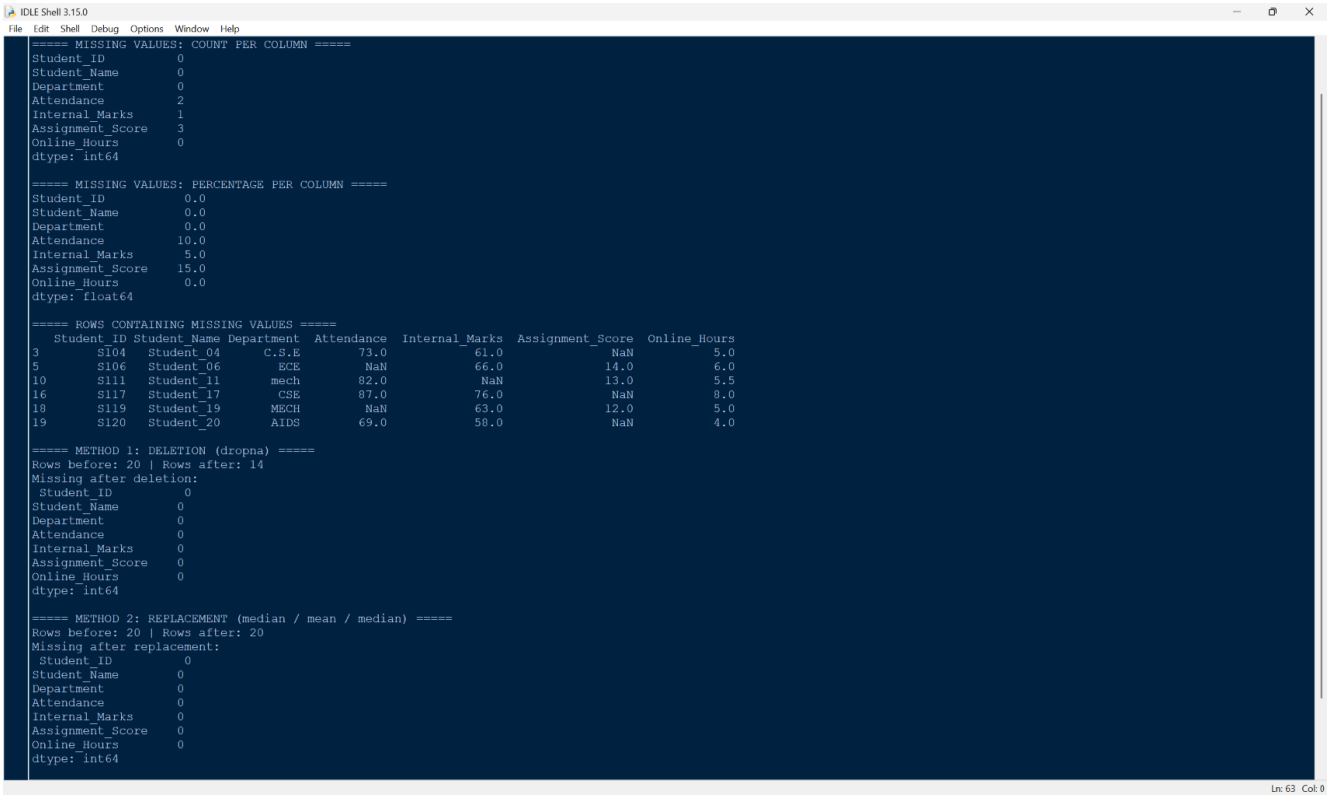
# ---------- METHOD 1 : DELETION ----------
before_rows = len(df)
df_drop = df.dropna(subset=["Attendance", "Internal_Marks", "Assignment_Score"])
print("\n===== METHOD 1: DELETION (dropna) =====")
print("Rows before:", before_rows, "| Rows after:", len(df_drop))
print("Missing after deletion:\n", df_drop.isnull().sum())

# ---------- METHOD 2 : REPLACEMENT ----------
df_fill = df.copy()
df_fill["Attendance"]       =
df_fill["Attendance"].fillna(df_fill["Attendance"].median())
df_fill["Internal_Marks"]   =
df_fill["Internal_Marks"].fillna(df_fill["Internal_Marks"].mean().round(1))
df_fill["Assignment_Score"] =
df_fill["Assignment_Score"].fillna(df_fill["Assignment_Score"].median())
print("\n===== METHOD 2: REPLACEMENT (median / mean / median) =====")
print("Rows before:", before_rows, "| Rows after:", len(df_fill))
print("Missing after replacement:\n", df_fill.isnull().sum())

print("\n===== BEFORE vs AFTER : MEAN & MEDIAN =====")
cols = ["Attendance", "Internal_Marks", "Assignment_Score"]
compare = pd.DataFrame({
    "Mean_Before":   df[cols].mean().round(2),
    "Mean_Deleted":  df_drop[cols].mean().round(2),
    "Mean_Filled":   df_fill[cols].mean().round(2),
    "Median_Before": df[cols].median(),
    "Median_Filled": df_fill[cols].median(),
})
```

```
print(compare)
```

Output – Screenshot 3



3.2 Missing-value summary

Column	Missing count	Percentage (20 students)	Students affected
Attendance	2	10%	S106, S119
Internal_Marks	1	5%	S111
Assignment_Score	3	15%	S104, S117, S120
Other columns	0	0%	–

Six students (S104, S106, S111, S117, S119, S120) have at least one missing value.

3.3 Method used for each column and justification

Column	Method selected	Justification
Attendance	Replace with the median (77.5)	Attendance is a percentage that can contain extreme or wrong entries (for example 105), and the median is not pulled by such values. The student keeps a typical value instead of being removed.
Internal_Marks	Replace with the mean (63.8)	Only one value (5%) is missing and the marks are spread fairly evenly (mean 63.84, median 66), so the mean is a reasonable substitute.
Assignment_Score	Replace with the median (14)	Three values (15%) are missing, too many to delete. Scores are small whole numbers out of 20, for which the median gives a realistic value.

3.4 Comparison before and after treatment

Measure	Before (with gaps)	After deletion	After replacement
Rows in dataset	20	14 (6 students lost)	20
Mean attendance	78.72	79.00	78.60
Mean internal marks	63.84	63.50	63.84
Mean assignment score	13.53	13.64	13.60
Missing values left	6 cells	0	0

Analysis. Deletion removes every row that has a gap, so the dataset shrinks from 20 to 14 students (30% of the records lost). Those six students may be exactly the ones who miss classes or tests, so deleting them can bias the study and removes useful information about their marks and online hours. Replacement keeps all 20 students and changes the column averages only slightly (for example mean attendance moves from 78.72 to 78.60). Replacement is therefore the better choice for this small dataset. Its limitation is that filled values are estimates, not observed facts, and they slightly reduce the natural variation in the data. The final pipeline in Task 6 uses the median for all three columns because outlier checks show that the median is the more robust choice.

TASK 4 – Sorting, Grouping and Ranking (5 Marks)

Organise the prepared student dataset to compare academic performance across students and departments.

4.1 Python code

This program uses the prepared file students_clean.csv, which is created by task6_preprocessing.py (run Task 6 first).

Python code *(file: task4_sort_group_rank.py)*

```
# TASK 4 - Sorting, Grouping and Ranking   (run task6_preprocessing.py first)
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_clean.csv")

print("===== (a) SORTED BY INTERNAL MARKS (DESCENDING) - TOP 10 =====")
by_marks = df.sort_values("Internal_Marks", ascending=False)
print(by_marks[["Student_ID", "Department", "Attendance", "Internal_Marks"]].head(10))

print("\n===== SORTED BY ATTENDANCE (DESCENDING) - TOP 10 =====")
by_att = df.sort_values("Attendance", ascending=False)
print(by_att[["Student_ID", "Department", "Attendance", "Internal_Marks"]].head(10))

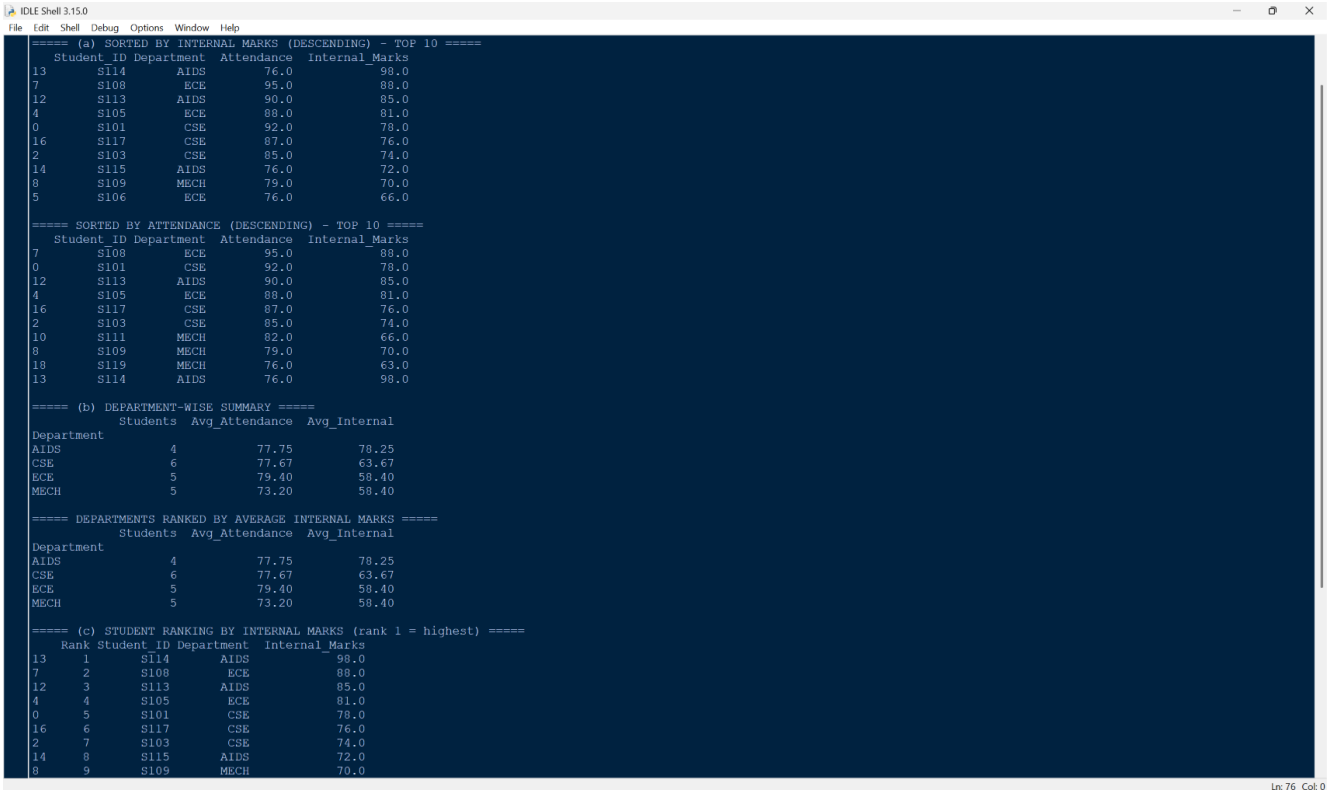
print("\n===== (b) DEPARTMENT-WISE SUMMARY =====")
dept = df.groupby("Department").agg(
    Students=("Student_ID", "count"),
    Avg_Attendance=("Attendance", "mean"),
    Avg_Internal=("Internal_Marks", "mean")
).round(2)
print(dept)

print("\n===== DEPARTMENTS RANKED BY AVERAGE INTERNAL MARKS =====")
print(dept.sort_values("Avg_Internal", ascending=False))

print("\n===== (c) STUDENT RANKING BY INTERNAL MARKS (rank 1 = highest) =====")
df["Rank"] = df["Internal_Marks"].rank(ascending=False, method="min").astype(int)
print(df.sort_values("Rank")[["Rank", "Student_ID", "Department", "Internal_Marks"]])

print("\n===== RANK 1 STUDENT IN EACH DEPARTMENT =====")
print(df.loc[df.groupby("Department")["Internal_Marks"].idxmax(), ["Department",
"Student_ID", "Internal_Marks"]])
```

Output – Screenshot 4



4.2 Department-wise summary

Department	Students	Average attendance (%)	Average internal marks
AIDS	4	77.75	78.25
CSE	6	77.67	63.67
ECE	5	79.40	58.40
MECH	5	73.20	58.40

4.3 Interpretation

- **Sorting.** Sorting with sort_values(ascending=False) places S114 (98), S108 (88) and S113 (85) at the top for internal marks, and S108 (95%), S101 (92%) and S113 (90%) at the top for attendance. S108 and S113 appear in both lists, which suggests that regular attendance and good marks go together.
- **Grouping.** groupby("Department") shows that AIDS has the best average marks (78.25). CSE is second (63.67). ECE and MECH tie at 58.40.
- **Attendance is not the whole story.** ECE has the highest average attendance (79.40) but only 58.40 average marks. The reason is Student_18 (S118), who has 74% attendance and only 12 marks. Without

that one record the ECE average would be about 70. MECH combines the lowest attendance (73.20, below the 75% requirement) with low marks, so it is the department that most needs attention.

- **Ranking.** rank(ascending=False, method="min") gives rank 1 to the highest marks. S106 and S111 both scored 66, so both receive rank 10 and the next student receives rank 12. The method argument decides how ties are handled. Department toppers are S114 (AIDS, 98), S108 (ECE, 88), S101 (CSE, 78) and S109 (MECH, 70), which also shows that the best MECH student is below the best student of every other department.

TASK 5 – Data Manipulation Using Pandas and NumPy (5 Marks)

Prepare useful variables from the student dataset for further analysis.

5.1 Python code

The program uses students_clean.csv (created by task6_preprocessing.py). Calculations used:

- **Total_Assessment** = Internal_Marks (out of 100) + Assignment_Score (out of 20), so the total is out of 120.
- **Assessment_Percent** = Total_Assessment ÷ 120 × 100.
- **Performance_Band**: up to 50% = Needs Support, above 50% up to 70% = Average, above 70% = Good.

Python code *(file: task5_manipulation.py)*

```
# TASK 5 - Data Manipulation Using Pandas and NumPy   (run task6_preprocessing.py first)
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_clean.csv")

# (a) PANDAS - derived columns
# Total_Assessment (out of 120) = Internal_Marks (out of 100) + Assignment_Score (out of 20)
df["Total_Assessment"] = df["Internal_Marks"] + df["Assignment_Score"]
# Assessment_Percent = Total_Assessment / 120 * 100
df["Assessment_Percent"] = (df["Total_Assessment"] / 120 * 100).round(2)
# Performance_Band using conditions
df["Performance_Band"] = pd.cut(df["Assessment_Percent"], bins=[0, 50, 70, 100],
                                labels=["Needs Support", "Average", "Good"])
print("===== (a) NEW DERIVED COLUMNS =====")
print(df[["Student_ID", "Internal_Marks", "Assignment_Score", "Total_Assessment",
          "Assessment_Percent", "Performance_Band"]].head(10))

# (b) NUMPY - numerical operations
marks = df["Internal_Marks"].to_numpy()
print("\n===== (b) NUMPY OPERATIONS ON INTERNAL MARKS =====")
print("Mean               :", np.mean(marks).round(2))
print("Median             :", np.median(marks))
print("Standard deviation :", np.std(marks).round(2))
print("Min / Max          :", np.min(marks), "/", np.max(marks))

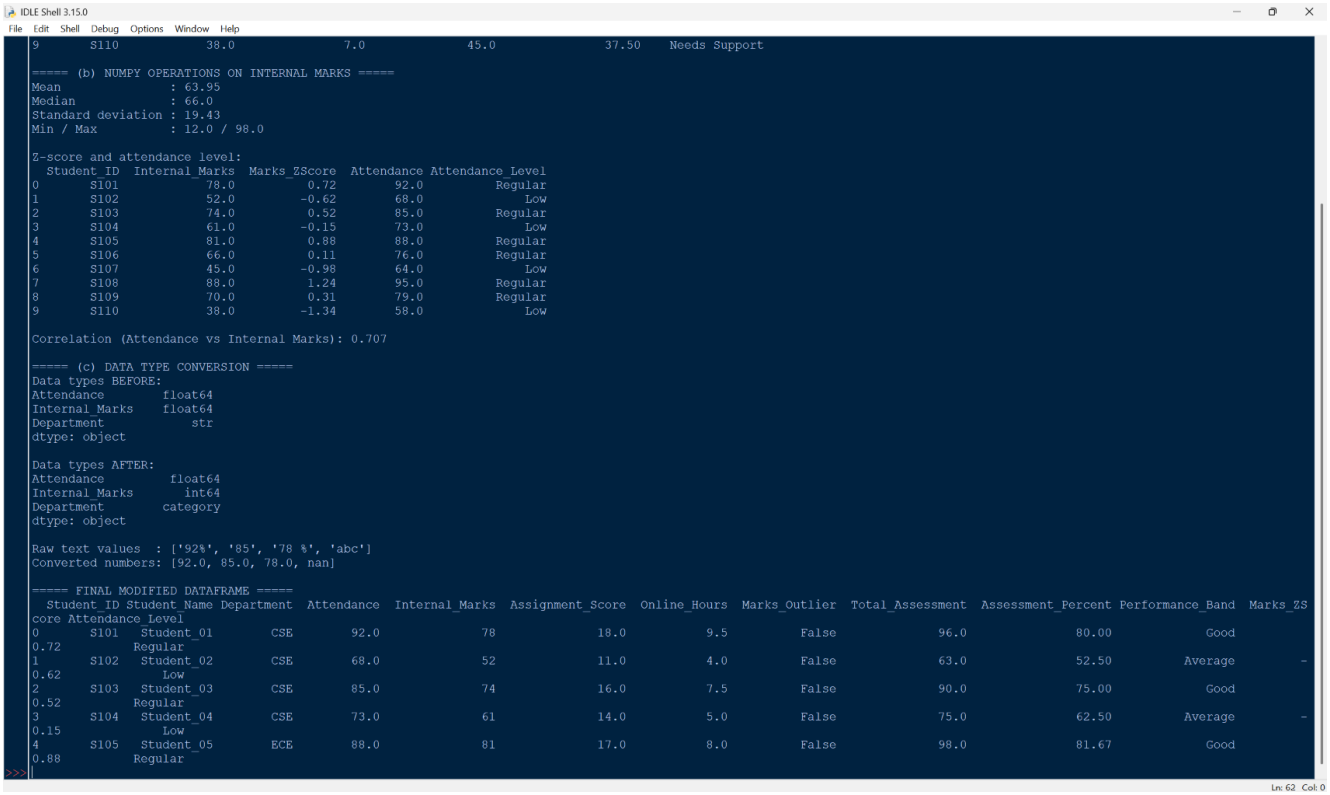
df["Marks_ZScore"] = np.round((marks - np.mean(marks)) / np.std(marks), 2)
df["Attendance_Level"] = np.where(df["Attendance"] >= 75, "Regular", "Low")
print("\nZ-score and attendance level:")
print(df[["Student_ID", "Internal_Marks", "Marks_ZScore", "Attendance", "Attendance_Level"]].head(10))
print("\nCorrelation (Attendance vs Internal Marks):",
      np.corrcoef(df["Attendance"], df["Internal_Marks"])[0, 1].round(3))

# (c) DATA TYPE CONVERSION
print("\n===== (c) DATA TYPE CONVERSION =====")
print("Data types BEFORE:")
print(df[["Attendance", "Internal_Marks", "Department"]].dtypes)
df["Internal_Marks"] = df["Internal_Marks"].round().astype(int)       # float -> int
df["Department"] = df["Department"].astype("category")               # text -> category
print("\nData types AFTER:")
print(df[["Attendance", "Internal_Marks", "Department"]].dtypes)

# raw-format example: converting a text value with a % sign
raw_values = pd.Series(["92%", "85", "78 %", "abc"])
print("\nRaw text values  :", raw_values.tolist())
print("Converted numbers:", pd.to_numeric(raw_values.str.replace("%", "").str.strip(),
errors="coerce").tolist())

print("\n===== FINAL MODIFIED DATAFRAME =====")
print(df.head())
```

Output – Screenshot 5



5.2 Interpretation of the transformations

Operation	Library	Result and purpose
Total_Assessment, Assessment_Percent	Pandas	Combines marks with different maximums into one comparable percentage. S101 gets 96/120 = 80%, S110 gets 45/120 = 37.5%.
Performance_Band (pd.cut)	Pandas	Converts a number into a label (Good / Average / Needs Support) that faculty can read at a glance. S107 and S110 fall in Needs Support.
np.mean, np.median, np.std, np.min, np.max	NumPy	Mean 63.95, median 66, standard deviation 19.43, range 12 to 98. The wide spread shows large differences between students.
Z-score = (mark − mean) ÷ std	NumPy	Shows how far each student is from the class average in standard-deviation units: S108 = +1.24 (well above), S110 = −1.34 (well below).
np.where (Attendance_Level)	NumPy	Labels each student Regular (≥ 75) or Low (< 75) for grouping and filtering.
np.corrcoef	NumPy	Correlation between attendance and marks is 0.707, a fairly strong positive relationship.
astype / pd.cut conversions	Pandas	Internal_Marks float → int, Department text → category (less memory, faster grouping). The text “92%” becomes 92.0 and the invalid text “abc” becomes NaN with errors="coerce", so bad entries do not crash the program.

Why this is useful. Derived columns turn raw entries into measures that match the question being asked (overall performance, support level and relative standing). NumPy operations work on whole columns at once, which is fast and avoids loops. Correct data types make later steps such as sorting, grouping and plotting reliable.

TASK 6 – Data Preprocessing Workflow (4 Marks)

Develop an ordered preprocessing workflow to convert the raw student dataset into a clean dataset suitable for visualization.

6.1 Python code

The program follows the order shown in the flowchart in section 6.2. At the end it saves the cleaned data as **students_clean.csv**, which is used by Tasks 4, 5, 7, 8 and 9.

Python code *(file: task6_preprocessing.py)*

```
# TASK 6 - Data Preprocessing Workflow  (also creates students_clean.csv used by Tasks 4, 5, 7, 9)
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

# STEP 1 - READ
df = pd.read_csv("students_raw.csv")
print("STEP 1  Raw data shape:", df.shape)

# STEP 2 - DETECT AND REMOVE DUPLICATES
print("\nSTEP 2  Duplicate rows found:", df.duplicated().sum())
print(df[df.duplicated(keep=False)])
df = df.drop_duplicates(subset="Student_ID", keep="first").reset_index(drop=True)
print("        Shape after removing duplicates:", df.shape)

# STEP 3 - STANDARDISE TEXT (inconsistent department names)
print("\nSTEP 3  Department labels BEFORE:", sorted(df["Department"].unique()))
df["Department"] = df["Department"].str.strip().str.upper()
dept_map = {"C.S.E": "CSE", "COMPUTER SCIENCE": "CSE", "MECHANICAL": "MECH"}
df["Department"] = df["Department"].replace(dept_map)
print("        Department labels AFTER :", sorted(df["Department"].unique()))

# STEP 4 - FIX DATA TYPES (Attendance stored as text like "92%")
df["Attendance"] = pd.to_numeric(df["Attendance"].astype(str).str.replace("%", ""), errors="coerce")
print("\nSTEP 4  Attendance data type now:", df["Attendance"].dtype)

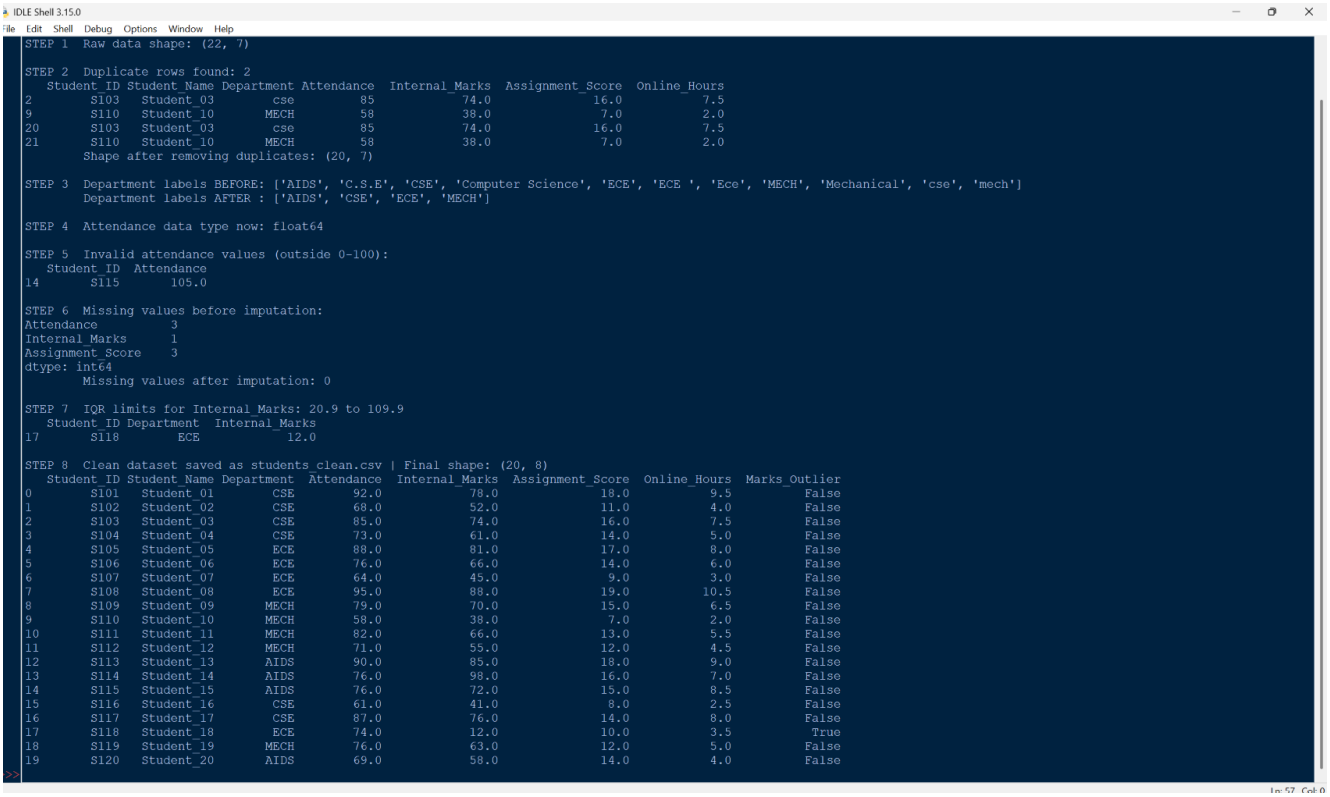
# STEP 5 - VALIDITY (RANGE) CHECKS
invalid_att = df[(df["Attendance"] < 0) | (df["Attendance"] > 100)]
print("\nSTEP 5  Invalid attendance values (outside 0-100):")
print(invalid_att[["Student_ID", "Attendance"]])
df.loc[(df["Attendance"] < 0) | (df["Attendance"] > 100), "Attendance"] = np.nan
df.loc[(df["Internal_Marks"] < 0) | (df["Internal_Marks"] > 100), "Internal_Marks"] = np.nan
df.loc[(df["Assignment_Score"] < 0) | (df["Assignment_Score"] > 20), "Assignment_Score"] = np.nan

# STEP 6 - HANDLE MISSING VALUES (median imputation)
print("\nSTEP 6  Missing values before imputation:")
print(df[["Attendance", "Internal_Marks", "Assignment_Score"]].isnull().sum())
for col in ["Attendance", "Internal_Marks", "Assignment_Score"]:
    df[col] = df[col].fillna(df[col].median())
print("        Missing values after imputation:", int(df.isnull().sum().sum()))

# STEP 7 - OUTLIER DETECTION (IQR rule) - flagged for review, not deleted
q1, q3 = df["Internal_Marks"].quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
df["Marks_Outlier"] = (df["Internal_Marks"] < low) | (df["Internal_Marks"] > high)
print(f"\nSTEP 7  IQR limits for Internal_Marks: {low:.1f} to {high:.1f}")
print(df[df["Marks_Outlier"]][["Student_ID", "Department", "Internal_Marks"]])

# STEP 8 - SAVE CLEAN DATASET
df.to_csv("students_clean.csv", index=False)
print("\nSTEP 8  Clean dataset saved as students_clean.csv | Final shape:", df.shape)
print(df)
```

Output – Screenshot 6



```
IDLE Shell 3.15.0
File  Edit  Shell  Debug  Options  Window  Help
STEP 1  Raw data shape: (22, 7)

STEP 2  Duplicate rows found: 2
   Student_ID Student_Name Department Attendance  Internal_Marks  Assignment_Score  Online_Hours
2        S103   Student_03        cse         85            74.0              16.0           7.5
9        S110   Student_10       MECH         58            38.0               7.0           2.0
20       S103   Student_03        cse         85            74.0              16.0           7.5
21       S110   Student_10       MECH         58            38.0               7.0           2.0
        Shape after removing duplicates: (20, 7)

STEP 3  Department labels BEFORE: ['AIDS', 'C.S.E', 'CSE', 'Computer Science', 'ECE', 'ECE ', 'Ece', 'MECH', 'Mechanical', 'cse', 'mech']
        Department labels AFTER : ['AIDS', 'CSE', 'ECE', 'MECH']

STEP 4  Attendance data type now: float64

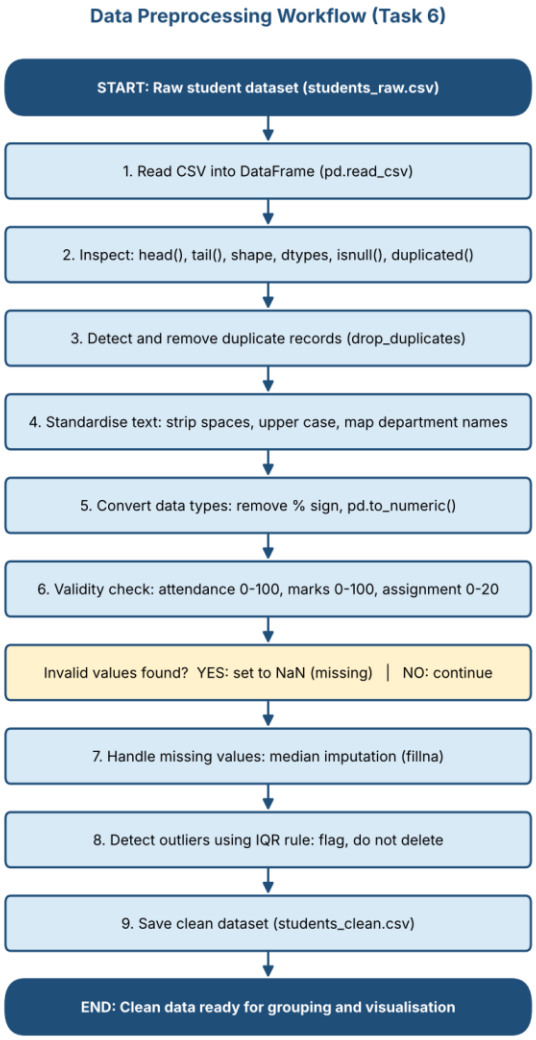
STEP 5  Invalid attendance values (outside 0-100):
   Student_ID  Attendance
14       S115       105.0

STEP 6  Missing values before imputation:
Attendance          3
Internal_Marks      1
Assignment_Score    3
dtype: int64
        Missing values after imputation: 0

STEP 7  IQR limits for Internal_Marks: 20.9 to 109.9
   Student_ID Department  Internal_Marks
17       S118        ECE            12.0

STEP 8  Clean dataset saved as students_clean.csv | Final shape: (20, 8)
   Student_ID Student_Name Department  Attendance  Internal_Marks  Assignment_Score  Online_Hours  Marks_Outlier
0        S101   Student_01        CSE        92.0            78.0              18.0           9.5          False
1        S102   Student_02        CSE        68.0            52.0              11.0           4.0          False
2        S103   Student_03        CSE        85.0            74.0              16.0           7.5          False
3        S104   Student_04        CSE        73.0            61.0              14.0           5.0          False
4        S105   Student_05        ECE        88.0            81.0              17.0           8.0          False
5        S106   Student_06        ECE        76.0            66.0              14.0           6.0          False
6        S107   Student_07        ECE        64.0            45.0               9.0           3.0          False
7        S108   Student_08        ECE        95.0            88.0              19.0          10.5          False
8        S109   Student_09       MECH        79.0            70.0              15.0           6.5          False
9        S110   Student_10       MECH        58.0            38.0               7.0           2.0          False
10       S111   Student_11       MECH        82.0            66.0              13.0           5.5          False
11       S112   Student_12       MECH        71.0            55.0              12.0           4.5          False
12       S113   Student_13       AIDS        90.0            85.0              18.0           9.0          False
13       S114   Student_14       AIDS        76.0            98.0              16.0           7.0          False
14       S115   Student_15       AIDS        76.0            72.0              15.0           8.5          False
15       S116   Student_16        CSE        61.0            41.0               8.0           2.5          False
16       S117   Student_17        CSE        87.0            76.0              14.0           8.0          False
17       S118   Student_18        ECE        74.0            12.0              10.0           3.5           True
18       S119   Student_19       MECH        76.0            63.0              12.0           5.0          False
19       S120   Student_20       AIDS        69.0            58.0              14.0           4.0          False
>>
                                                                                                    Ln: 57  Col: 0
```

6.2 Labelled preprocessing flowchart



6.3 Detection and correction of problems

Step	Problem	How it was detected	Correction	Result
3	Duplicate records	df.duplicated() and duplicated(keep=False)	drop_duplicates(subset="Student_ID")	S103 and S110 repeats removed: 22 → 20 rows
4	Inconsistent department names	Compare unique() before and after	strip(), upper() and a mapping dictionary	11 labels → 4 (AIDS, CSE, ECE, MECH)
5	Attendance stored as text	dtype was text; values such as “92%”	Remove % and pd.to_numeric()	Attendance becomes float64
6	Invalid value	Range check: attendance outside 0–100	Set to NaN, then fill	S115 (105%) replaced by 76
7	Missing values	isnull().sum()	fillna(median)	Attendance 3, marks 1, assignment 3 → 0 missing
8	Unusual mark	IQR rule: limits 20.9 to 109.9	Flagged in Marks_Outlier, kept	S118 (12 marks) flagged for review

Filled values (the medians) were attendance 76, internal marks 66 and assignment score 14. The invalid attendance of 105 was counted with the three missing attendance values, so the attendance column received three replacements.

6.4 How the workflow improves reliability

Aspect	Raw data	Clean data
Rows	22 (2 repeated students)	20 unique students
Department labels	11 spellings	4 consistent labels
Attendance type	Text	Numeric
Missing / invalid cells	6 missing + 1 invalid	0
Unusual marks	Not identified	Flagged for review

The order of the steps matters. Duplicates are removed before filling missing values so that repeated students do not influence the median. Text is standardised before grouping so that one department is not split into several groups. Types are fixed before range checks so that numeric comparisons work. Invalid values are converted to missing before imputation so that wrong values do not spread into the results. After these steps averages, counts, rankings and plots describe each real student exactly once, which makes later conclusions trustworthy.

TASK 7 – Data Visualization Using Matplotlib (6 Marks)

Create suitable visual representations to understand the distribution and patterns in student performance.

7.1 Python code

The program (which uses students_clean.csv) draws three plots one after another. Each plot opens in its own window; take the screenshot, close the window and the next plot appears. Each plot is also saved as a PNG file in the same folder.

Python code *(file: task7_visualization.py)*

```
# TASK 7 - Data Visualization Using Matplotlib   (run task6_preprocessing.py first)
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

import matplotlib.pyplot as plt

df = pd.read_csv("students_clean.csv")

# ---------- PLOT 1 : BAR CHART - average internal marks by department ----------
avg_marks = df.groupby("Department")["Internal_Marks"].mean().round(2)
print("Average internal marks by department:\n", avg_marks)

plt.figure(figsize=(7, 5))
bars = plt.bar(avg_marks.index, avg_marks.values, color=["#2E74B5", "#5B9BD5", "#9DC3E6", "#1F4E79"],
label="Average Internal Marks")
for b in bars:
    plt.text(b.get_x() + b.get_width() / 2, b.get_height() + 1, f"{b.get_height():.1f}", ha="center")
plt.title("Average Internal Marks by Department")
plt.xlabel("Department")
plt.ylabel("Average Internal Marks (out of 100)")
plt.ylim(0, 100)
plt.legend()
plt.tight_layout()
plt.savefig("plot1_bar_department.png", dpi=150)
plt.show()

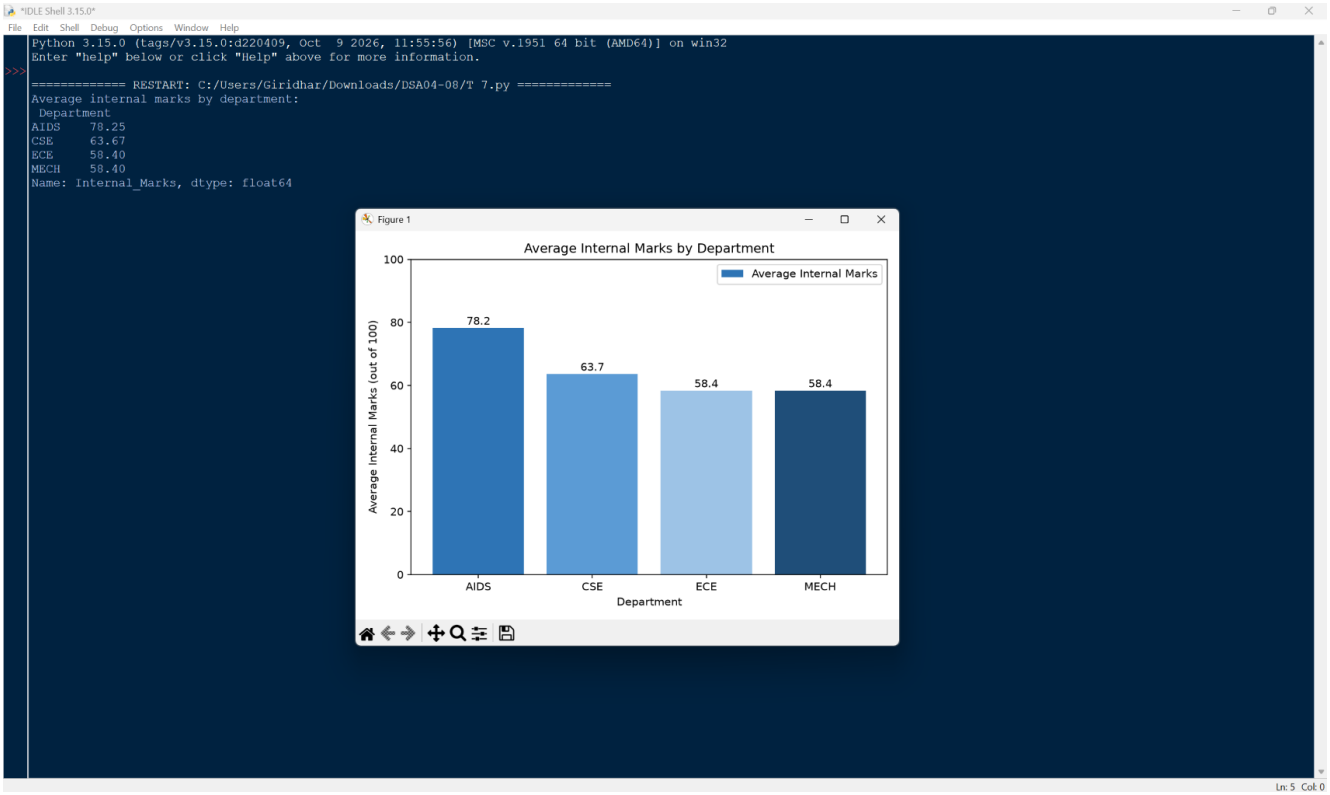
# ---------- PLOT 2 : HISTOGRAM - distribution of attendance ----------
plt.figure(figsize=(7, 5))
plt.hist(df["Attendance"], bins=[55, 62, 69, 76, 83, 90, 97], color="#5B9BD5", edgecolor="black",
label="Students")
plt.axvline(75, color="red", linestyle="--", label="75% attendance limit")
plt.title("Distribution of Attendance Percentage")
plt.xlabel("Attendance (%)")
plt.ylabel("Number of Students")
plt.legend()
plt.tight_layout()
plt.savefig("plot2_hist_attendance.png", dpi=150)
plt.show()

# ---------- PLOT 3 : SCATTER PLOT - attendance vs internal marks ----------
plt.figure(figsize=(7, 5))
colors = {"CSE": "#1F77B4", "ECE": "#FF7F0E", "MECH": "#2CA02C", "AIDS": "#D62728"}
for dept, grp in df.groupby("Department"):
    plt.scatter(grp["Attendance"], grp["Internal_Marks"], s=70, color=colors[dept], label=dept)
m, c = np.polyfit(df["Attendance"], df["Internal_Marks"], 1)       # trend line
x = np.linspace(df["Attendance"].min(), df["Attendance"].max(), 50)
plt.plot(x, m * x + c, color="black", linestyle="--", label="Trend line")
plt.title("Attendance vs Internal Marks")
plt.xlabel("Attendance (%)")
plt.ylabel("Internal Marks (out of 100)")
plt.legend(title="Department")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("plot3_scatter_attendance_marks.png", dpi=150)
plt.show()

print("\nCorrelation between attendance and internal marks:",
      round(df["Attendance"].corr(df["Internal_Marks"]), 3))
```

7.2 Plot 1 – Bar chart: average internal marks by department

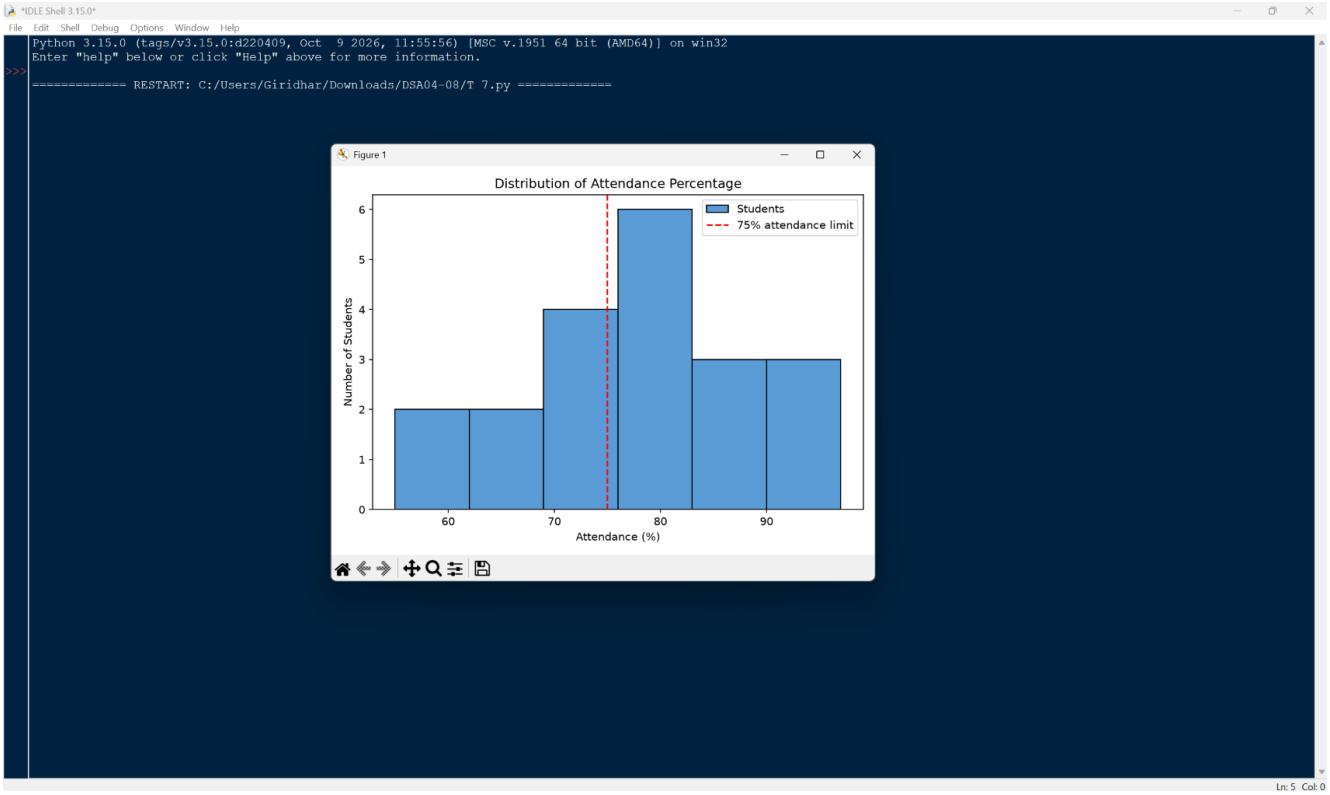
Output – Screenshot 7



Interpretation. AIDS has the highest average (78.25), followed by CSE (63.67). ECE and MECH are equal at 58.40, almost 20 marks below AIDS. The chart shows quickly which departments are lagging, but it hides the spread inside each department (for example one very low ECE mark), so it should be read together with the other plots.

7.3 Plot 2 – Histogram: distribution of attendance

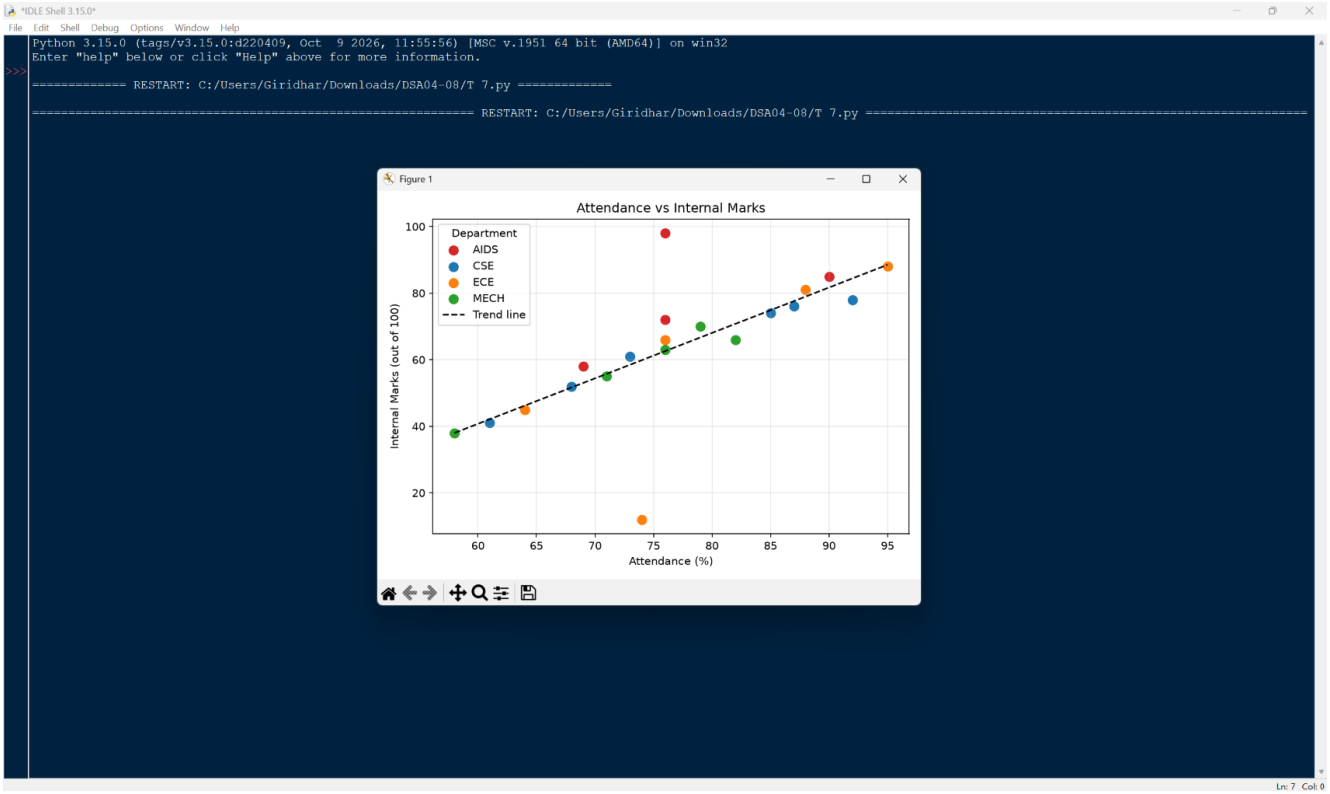
Output – Screenshot 8



Interpretation. The six bins hold 2, 2, 4, 6, 3 and 3 students (from 55–62% up to 90–97%). The tallest bar is 76–83% with six students. Eight students lie to the left of the red 75% line, which is 40% of the class, and the distribution has a longer tail towards low attendance. Part of the peak at 76% comes from missing or invalid values that were replaced by the median, which is a limitation of median filling.

7.4 Plot 3 – Scatter plot: attendance vs internal marks

Output – Screenshot 9



Interpretation. The points rise from the lower left to the upper right and the trend line slopes upwards: students who attend more tend to obtain higher marks (correlation 0.707). The students with 58–64% attendance have the lowest marks (38–45). One ECE student (74% attendance, 12 marks) lies far below the trend line, so his or her low marks cannot be explained by attendance alone and should be examined individually. The correlation shows association, not proof that attendance causes higher marks.

TASK 8 – Choosing Suitable Python Libraries (4 Marks)

The college team plans to use Python libraries for different stages of data preparation and analysis.

8.1 Roles of the libraries in the student-dataset context

Library	Role	Use with the student dataset
Pandas	Tables (DataFrames) for reading, selecting, cleaning, grouping and summarising data	Reads students_raw.csv, selects columns, filters low attendance, fills missing values, groups by department
NumPy	Fast numerical arrays and mathematical functions	Computes mean, standard deviation and z-scores of marks, np.where labels, correlation, percentile
SciPy	Scientific and statistical routines	stats.pearsonr tests whether the attendance–marks relationship is statistically significant
Scikit-learn	Machine-learning tools and data scaling	MinMaxScaler brings attendance and marks to a 0–1 scale; later clustering or prediction of at-risk students
Matplotlib	Charts and plots	Bar chart by department, histogram of attendance, scatter plot of attendance against marks

8.2 Mapping of activities to libraries

Activity	Suitable library	Relevant function / capability
Reading data	Pandas	pd.read_csv(), head(), tail(), info()
Selecting and filtering	Pandas	df[[cols]], boolean filtering, loc / iloc
Cleaning (missing, duplicate, text)	Pandas	isnull(), fillna(), dropna(), drop_duplicates(), str.strip(), replace()
Sorting, grouping, ranking	Pandas	sort_values(), groupby().agg(), rank()
Numerical operations	NumPy	np.mean(), np.std(), np.where(), np.corrcoef(), np.percentile()
Statistical testing	SciPy	scipy.stats.pearsonr(), ttest_ind()
Scaling / modelling	Scikit-learn	MinMaxScaler, StandardScaler, KMeans, LogisticRegression
Visualization	Matplotlib	plt.bar(), plt.hist(), plt.scatter(), plt.boxplot(), savefig()

8.3 Python code (demonstration of the libraries)

Python code *(file: task8_libraries_demo.py)*

```
# TASK 8 - Choosing Suitable Python Libraries  (small demonstration of each library)
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

import matplotlib
import scipy
import sklearn
from scipy import stats
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt

print("Library versions -> pandas", pd.__version__, "| numpy", np.__version__,
      "| scipy", scipy.__version__, "| scikit-learn", sklearn.__version__,
      "| matplotlib", matplotlib.__version__)

df = pd.read_csv("students_clean.csv")

# PANDAS - reading, selecting, cleaning, grouping
print("\n[Pandas ] Average marks by department:")
print(df.groupby("Department")["Internal_Marks"].mean().round(2))

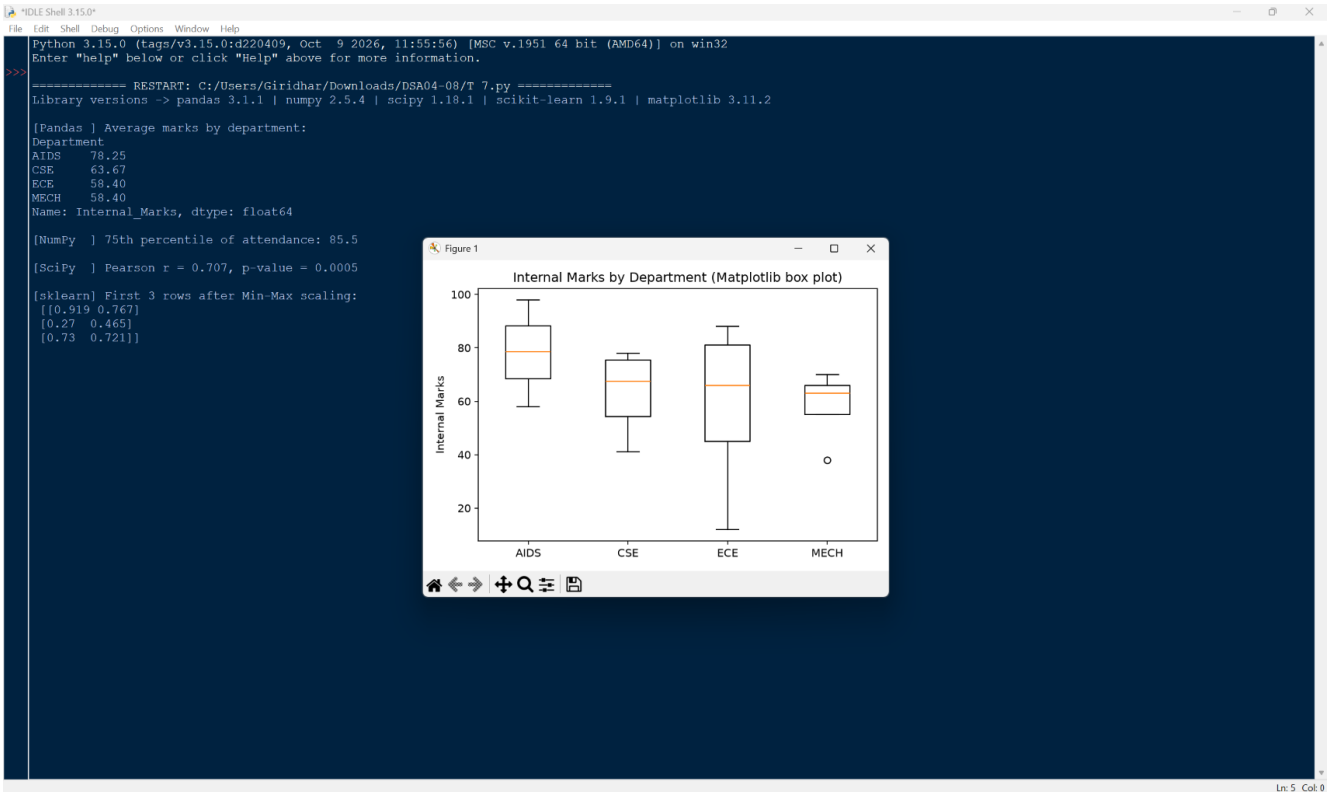
# NUMPY - numerical operation
print("\n[NumPy  ] 75th percentile of attendance:", np.percentile(df["Attendance"], 75))
```

```
# SCIPY - statistical test (Pearson correlation with p-value)
r, p = stats.pearsonr(df["Attendance"], df["Internal_Marks"])
print(f"\n[SciPy  ] Pearson r = {r:.3f}, p-value = {p:.4f}")

# SCIKIT-LEARN - scaling the numeric columns to 0-1
scaled = MinMaxScaler().fit_transform(df[["Attendance", "Internal_Marks"]])
print("\n[sklearn] First 3 rows after Min-Max scaling:\n", scaled[:3].round(3))

# MATPLOTLIB - visualisation
plt.figure(figsize=(6, 4))
depts = sorted(df.Department.unique())
plt.boxplot([df[df.Department == d]["Internal_Marks"] for d in depts])
plt.xticks(range(1, len(depts) + 1), depts)
plt.title("Internal Marks by Department (Matplotlib box plot)")
plt.ylabel("Internal Marks")
plt.tight_layout()
plt.savefig("plot_task8_boxplot.png", dpi=150)
plt.show()
```

Output – Screenshot 10



8.4 Analysis

The demonstration shows each library doing the job it is best at: Pandas gives the department averages, NumPy gives the 75th-percentile attendance (85.5), SciPy gives a Pearson correlation of 0.707 with a p-value of about 0.0005 (so the relationship is unlikely to be chance), Scikit-learn scales the columns to 0–1 and Matplotlib draws the box plot. Choosing the right library improves efficiency because each tool is optimised for its task: Pandas handles labelled tables with one-line operations instead of manual loops, NumPy computes on entire arrays quickly

, SciPy supplies tested statistical tests instead of hand-written formulas, and Matplotlib turns results into visuals. Using them together keeps the code short, readable and less error-prone, which matters when the same preparation must be repeated for every semester.

TASK 9 – Interpreting Visualized Data for Academic Support (6 Marks)

The department wants to use the prepared dataset and visualizations to identify students who may need academic support.

9.1 Python code

A student is marked **at risk** when attendance is below 75% **and** internal marks are below 60 (thresholds chosen for illustration).

Python code *(file: task9_interpretation.py)*

```
# TASK 9 - Interpreting Visualized Data for Academic Support   (run task6_preprocessing.py first)
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

import matplotlib.pyplot as plt

df = pd.read_csv("students_clean.csv")

# ---------- PATTERN 1 : low attendance together with low marks ----------
df["At_Risk"] = (df["Attendance"] < 75) & (df["Internal_Marks"] < 60)
print("===== PATTERN 1: AT-RISK STUDENTS (attendance < 75 AND marks < 60) =====")
risk = df[df["At_Risk"]][["Student_ID", "Department", "Attendance", "Internal_Marks", "Assignment_Score"]]
print(risk.sort_values("Internal_Marks"))
print("Students at risk:", len(risk), "out of", len(df))

print("\nAverage marks - Low attendance (<75) vs Regular attendance (>=75):")
df["Attendance_Group"] = np.where(df["Attendance"] < 75, "Low (<75%)", "Regular (>=75%)")
print(df.groupby("Attendance_Group")[["Internal_Marks", "Assignment_Score", "Online_Hours"]].mean().round(2))

# ---------- PATTERN 2 : department differences ----------
print("\n===== PATTERN 2: DEPARTMENT-WISE AVERAGES AND AT-RISK COUNT =====")
summary = df.groupby("Department").agg(
    Students=("Student_ID", "count"),
    Avg_Attendance=("Attendance", "mean"),
    Avg_Marks=("Internal_Marks", "mean"),
    At_Risk=("At_Risk", "sum")
).round(2)
print(summary)

# ---------- VISUALISATION 1 : scatter with risk zone ----------
plt.figure(figsize=(7, 5))
ok = df[~df["At_Risk"]]
plt.scatter(ok["Attendance"], ok["Internal_Marks"], color="#2E74B5", s=70, label="Other students")
plt.scatter(risk["Attendance"], risk["Internal_Marks"], color="red", s=90, marker="X", label="At-risk
students")
plt.axvline(75, color="grey", linestyle="--")
plt.axhline(60, color="grey", linestyle="--")
plt.title("At-Risk Zone: Low Attendance and Low Internal Marks")
plt.xlabel("Attendance (%)")
plt.ylabel("Internal Marks (out of 100)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("plot9_risk_scatter.png", dpi=150)
plt.show()

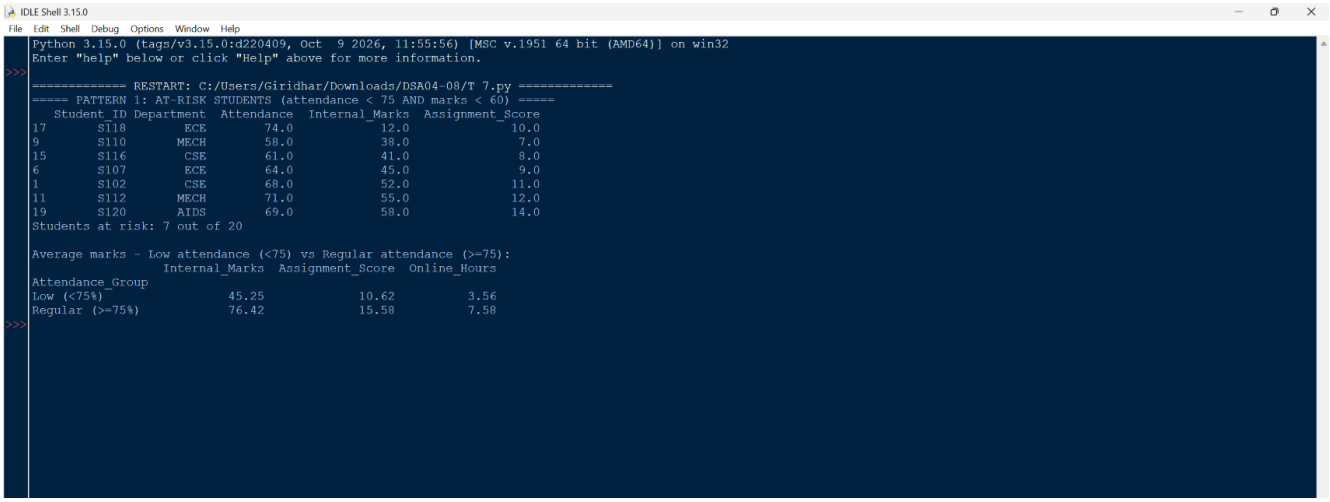
# ---------- VISUALISATION 2 : at-risk students per department ----------
plt.figure(figsize=(7, 5))
plt.bar(summary.index, summary["At_Risk"], color="#C0504D", label="At-risk students")
plt.title("Number of At-Risk Students by Department")
plt.xlabel("Department")
plt.ylabel("Number of Students")
plt.legend()
plt.tight_layout()
plt.savefig("plot9_risk_by_department.png", dpi=150)
plt.show()
```

9.2 Pattern 1 – Low attendance goes with low marks and low engagement

Group	Students	Average internal marks	Average assignment score	Average online hours
Low attendance (< 75%)	8	45.25	10.62	3.56
Regular attendance (≥ 75%)	12	76.42	15.58	7.58

Students with low attendance score about 31 marks less on average, submit weaker assignments and spend less than half the online time. Seven of the 20 students (35%) fall in the at-risk zone: S118, S110, S116, S107, S102, S112 and S120.

Output – Screenshot 12

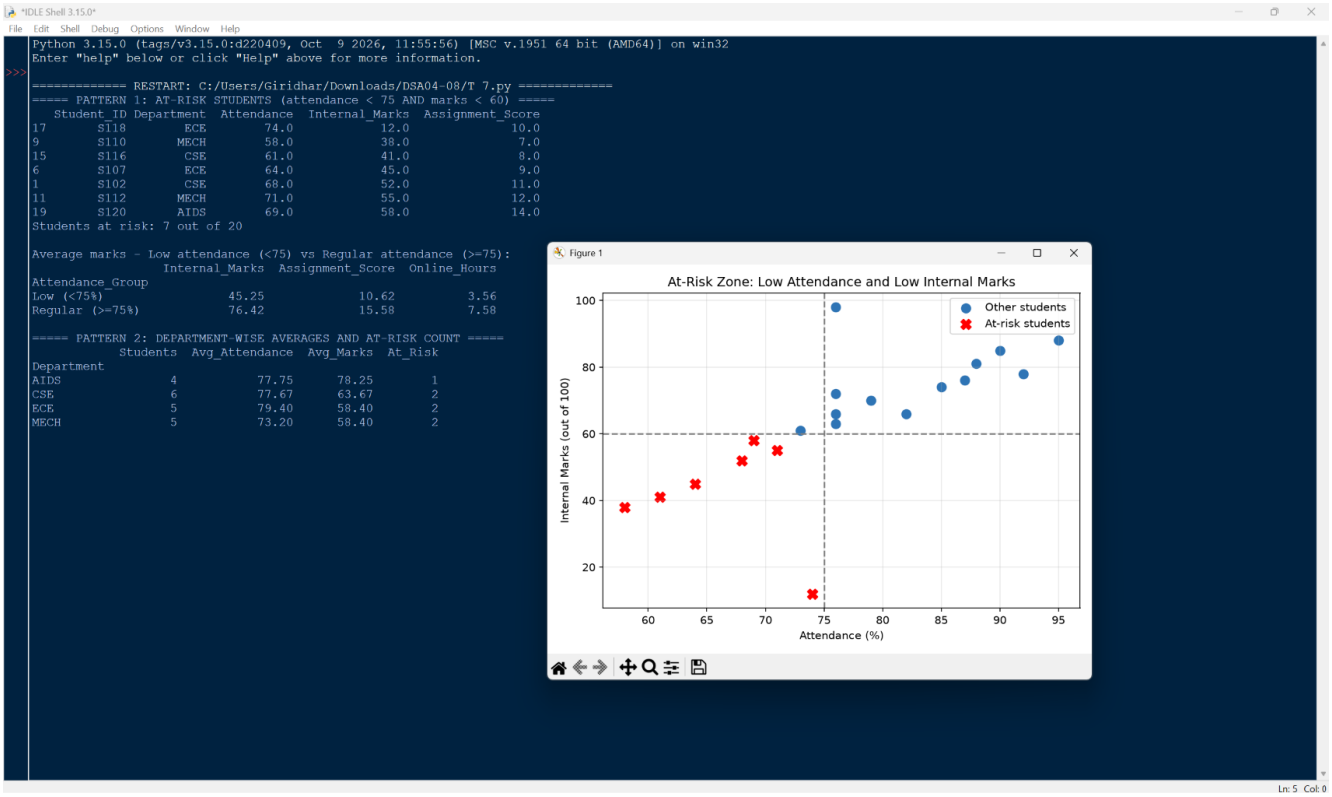


9.3 Pattern 2 – Departments differ in performance and risk

Department	Students	Avg attendance	Avg marks	At-risk students
AIDS	4	77.75	78.25	1
CSE	6	77.67	63.67	2
ECE	5	79.40	58.40	2
MECH	5	73.20	58.40	2

AIDS performs best and has only one at-risk student. MECH is the only department whose average attendance (73.20) is below 75% and it also has the lowest weekly online time (4.7 hours). ECE and CSE each have two at-risk students, and the ECE result is strongly affected by one very low mark.

Output – Screenshot 13



9.4 How the visualizations help communicate the findings

The scatter plot lets faculty see the risk zone at a glance: the red crosses in the lower-left quadrant are the students who need action, and the dashed lines show the thresholds used, so the rule is transparent. The department bar chart compares groups without reading a table, so a head of department can immediately see where support is concentrated. Visuals therefore turn 20 rows of numbers into a message that can be understood and discussed in a short meeting. The student list that comes with the chart gives the names (coded IDs) that mentors need for follow-up.

9.5 Recommended academic support actions

- **Targeted mentoring and remedial classes for the seven at-risk students.** Start with the four who have both very low attendance and very low marks (S118, S110, S116 and S107). Mentors meet each student, find the reasons (health, travel, understanding of topics) and set weekly targets for attendance and assignment submission. Progress is checked after the next internal test.
- **Department-level attendance and engagement monitoring, starting with MECH.** Introduce a weekly attendance alert when a student drops below 75%, and encourage online practice through short LMS quizzes, since low online hours accompany low marks. Share the teaching practices of the AIDS department with ECE and MECH, and review results again at the next assessment.

TASK 10 – Integrated Preprocessing and Visualization Case Study (7 Marks)

Consider a dataset containing student ID, department, attendance percentage, internal marks, assignment score and online learning activity. The dataset has missing attendance values, duplicate student records, inconsistent department names and a few unusually high or low marks. Analyse how the college can prepare this dataset and present meaningful findings.

10.1 Steps and assumptions

Step	Action	Pandas / Matplotlib tool
1. Read	Load the raw file	pd.read_csv()
2. Inspect	Check size, missing values, duplicates and department labels	isnull().sum(), duplicated(), unique()
3. Select	Keep only the six columns needed for the analysis (names are dropped)	df[cols]
4. Clean	Remove duplicates, standardise department names, convert attendance to numbers, treat invalid values, fill missing values, flag unusual marks	drop_duplicates(), str methods, to_numeric(), fillna(), IQR rule
5. Summarise	Sort, group and rank students and departments; compute correlation	sort_values(), groupby(), rank(), corr()
6. Visualise	Bar chart and scatter plot	plt.bar(), plt.scatter()

Assumptions: rows with the same Student_ID are duplicates and the first is kept; attendance above 100 is invalid and treated as missing; missing values are replaced by the median because it is not affected by extreme marks; unusual marks (found by the IQR rule) are flagged but retained because a very low mark may be a genuine result; and the department names CSE, C.S.E and Computer Science, ECE and Ece, and MECH and Mechanical refer to the same departments.

10.2 Python code (connected workflow)

Python code *(file: task10_case_study.py)*

```
# TASK 10 - Integrated Preprocessing and Visualization Case Study
# One connected workflow: read -> inspect -> select -> clean -> summarise -> visualise
import pandas as pd
import numpy as np
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

import matplotlib.pyplot as plt

# ===== STEP 1 : READ =====
df = pd.read_csv("students_raw.csv")
print("STEP 1 - READ: raw shape =", df.shape)

# ===== STEP 2 : INSPECT =====
print("\nSTEP 2 - INSPECT")
print("Missing values:\n", df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
print("Department labels:", sorted(df["Department"].unique()))

# ===== STEP 3 : SELECT the columns needed =====
cols = ["Student_ID", "Department", "Attendance", "Internal_Marks", "Assignment_Score", "Online_Hours"]
df = df[cols]
print("\nSTEP 3 - SELECT: columns kept ->", cols)

# ===== STEP 4 : CLEAN =====
# Assumptions: (1) identical Student_ID rows are duplicates, keep the first
#              (2) valid attendance 0-100, marks 0-100, assignment 0-20
#              (3) invalid values are treated as missing; missing values -> median
#              (4) unusual marks are flagged (IQR rule) but NOT deleted - they may be genuine
df = df.drop_duplicates(subset="Student_ID").reset_index(drop=True)
df["Department"] = (df["Department"].str.strip().str.upper()
                    .replace({"C.S.E": "CSE", "COMPUTER SCIENCE": "CSE", "MECHANICAL": "MECH"}))
df["Attendance"] = pd.to_numeric(df["Attendance"].astype(str).str.replace("%", ""), errors="coerce")
```

```
df.loc[df["Attendance"] > 100, "Attendance"] = np.nan
for col in ["Attendance", "Internal_Marks", "Assignment_Score"]:
    df[col] = df[col].fillna(df[col].median())
q1, q3 = df["Internal_Marks"].quantile([0.25, 0.75])
low, high = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
df["Marks_Outlier"] = (df["Internal_Marks"] < low) | (df["Internal_Marks"] > high)
print("\nSTEP 4 - CLEAN: shape =", df.shape, "| missing =", int(df.isnull().sum().sum()))
print("Departments now:", sorted(df["Department"].unique()))
print("Outliers flagged:\n", df[df["Marks_Outlier"]][["Student_ID", "Internal_Marks"]])

# ===== STEP 5 : SORT / GROUP / RANK =====
summary = df.groupby("Department").agg(
    Students=("Student_ID", "count"),
    Avg_Attendance=("Attendance", "mean"),
    Avg_Marks=("Internal_Marks", "mean"),
    Avg_Online_Hours=("Online_Hours", "mean")
).round(2).sort_values("Avg_Marks", ascending=False)
print("\nSTEP 5 - DEPARTMENT SUMMARY (sorted by average marks):")
print(summary)
df["Rank"] = df["Internal_Marks"].rank(ascending=False, method="min").astype(int)
print("\nTop 5 students:")
print(df.sort_values("Rank").head(5)[["Rank", "Student_ID", "Department", "Internal_Marks"]])
print("\nStudents needing attention (attendance < 75):")
print(df[df["Attendance"] < 75][["Student_ID", "Department", "Attendance", "Internal_Marks"]])
print("\nCorrelation matrix:")
print(df[["Attendance", "Internal_Marks", "Assignment_Score", "Online_Hours"]].corr().round(2))

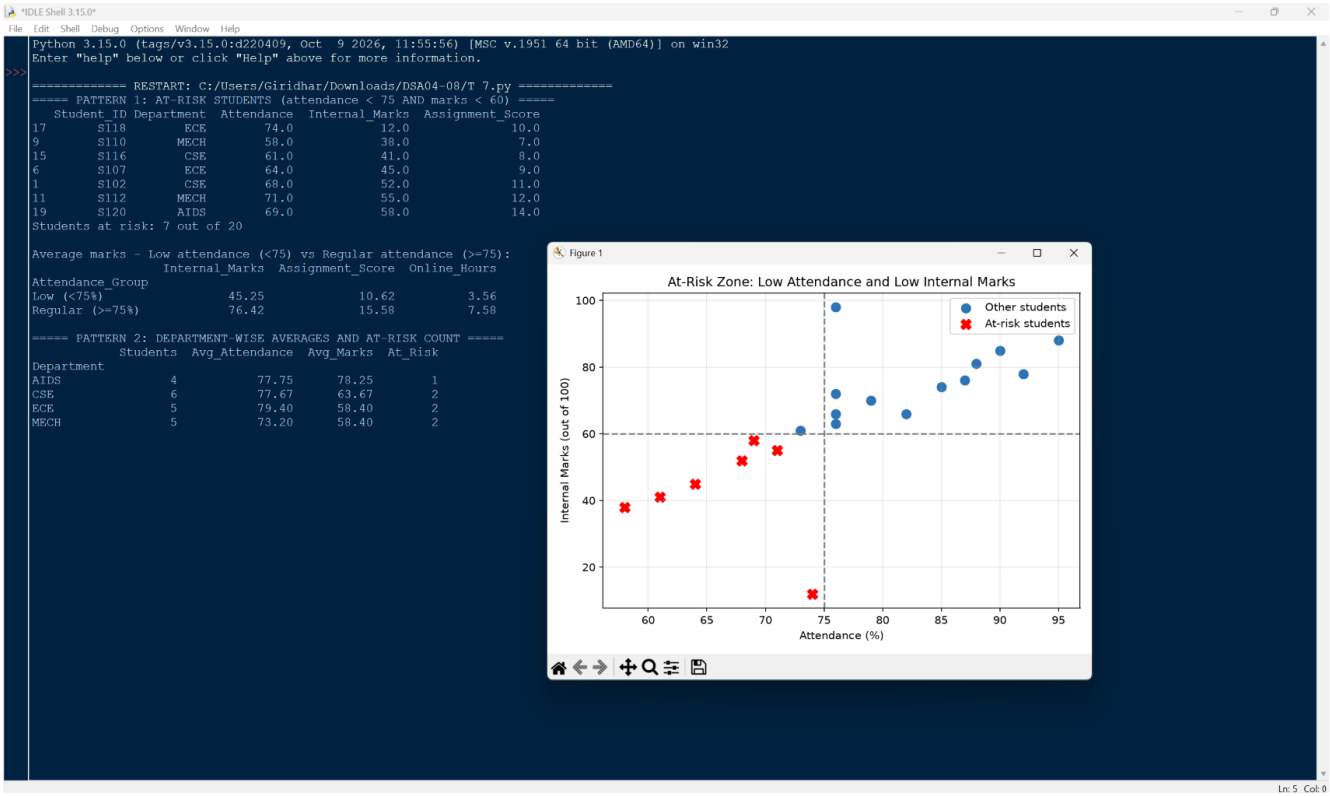
# ===== STEP 6 : VISUALISE =====
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].bar(summary.index, summary["Avg_Marks"], color="#2E74B5", label="Average marks")
axes[0].set_title("Average Internal Marks by Department")
axes[0].set_xlabel("Department")
axes[0].set_ylabel("Average Internal Marks")
axes[0].legend()

axes[1].scatter(df["Online_Hours"], df["Internal_Marks"], color="#C0504D", s=70, label="Students")
m, c = np.polyfit(df["Online_Hours"], df["Internal_Marks"], 1)
x = np.linspace(df["Online_Hours"].min(), df["Online_Hours"].max(), 50)
axes[1].plot(x, m * x + c, "k--", label="Trend line")
axes[1].set_title("Online Learning Hours vs Internal Marks")
axes[1].set_xlabel("Online Hours per Week")
axes[1].set_ylabel("Internal Marks")
axes[1].legend()

plt.tight_layout()
plt.savefig("plot10_integrated.png", dpi=150)
plt.show()
```

Output – Screenshot 14



10.3 Results

Cleaning outcome. The raw file (22 rows) became 20 unique students with 4 consistent department labels and no missing values. One unusual low mark (S118, 12) was flagged.

Department	Students	Avg attendance	Avg marks	Avg online hours
AIDS	4	77.75	78.25	7.12
CSE	6	77.67	63.67	6.08
ECE	5	79.40	58.40	6.20
MECH	5	73.20	58.40	4.70

The five top-ranked students are S114 (98), S108 (88), S113 (85), S105 (81) and S101 (78). Eight students have attendance below 75%: S102, S104, S107, S110, S112, S116, S118 and S120.

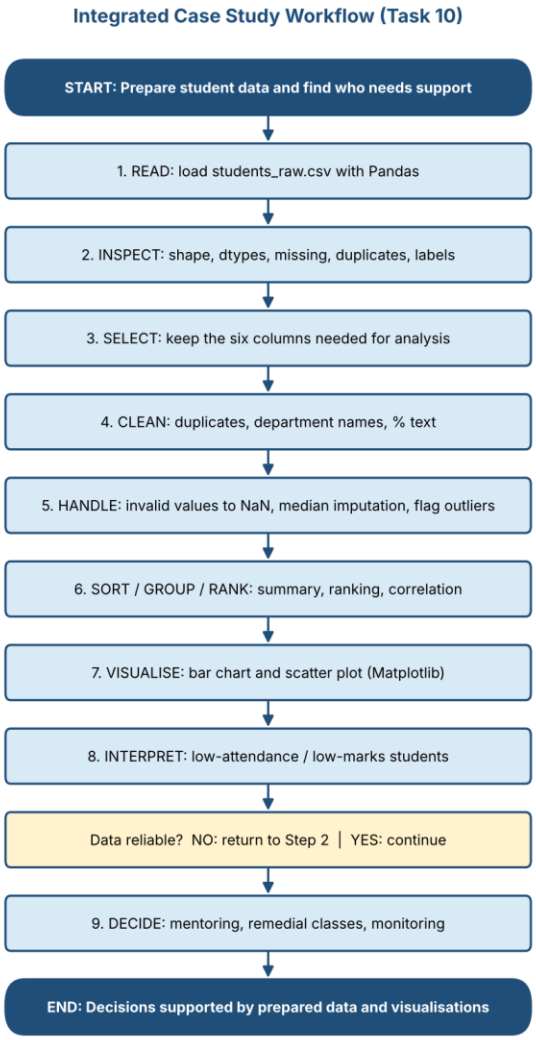
Pair of variables	Correlation
Attendance and Internal marks	0.71
Online hours and Internal marks	0.83
Assignment score and Internal marks	0.85
Online hours and Attendance	0.93

10.4 Interpretation of the visualizations

- **Bar chart.** AIDS has the highest average marks (78.25) and ECE and MECH the lowest (58.40), so support should be focused on these two departments.

- **Scatter plot.** Students who spend more weekly hours on the online platform generally obtain higher marks (correlation 0.83) and the trend line rises steadily. The students with 2–3.5 online hours have the lowest marks. This association should not be taken as proof of cause: attentive students may simply use the platform more.

10.5 Labelled workflow flowchart



10.6 How the prepared data supports academic decision-making

Before preparation, the dataset would have given wrong answers: duplicate students counted twice, one department appearing as several groups, text attendance that cannot be compared and impossible values affecting averages. After preparation, the same data gives reliable department averages, a ranked student list and a clear relationship between engagement and marks. The college can use these results to (1) arrange mentoring and remedial classes for the eight low-attendance students, especially those whose marks are also low, (2) review the teaching and attendance monitoring in MECH and ECE, and (3) encourage regular online learning because it is linked with better marks. The flowchart makes the process repeatable, so the same steps can be run on new data every semester and the effect of the actions can be checked.

OVERALL CONCLUSION

This report has shown how a raw student dataset containing missing values, duplicate records, inconsistent department names, wrong data types and unusual marks can be converted into a clean dataset using Pandas and NumPy. Reading, inspection, selection, filtering, sorting, grouping and ranking turned the data into meaningful summaries, and Matplotlib plots made the patterns easy to see: higher attendance and more online activity go with higher internal marks, and a small group of students combines low attendance with low marks.

The analysis also showed that each preprocessing choice affects the result. Deletion would have lost 30% of the students, while median replacement kept all records with little change to the averages. The central lesson is that analysis is only as reliable as the data preparation behind it, and that results from a small sample dataset should be treated as indicators for follow-up by faculty, not as final judgements about individual students.

REFERENCES / LEARNING RESOURCES

Course material: DSA04 – Fundamentals of Data Science, CO2 topics and assignment brief.

Library documentation: Pandas, NumPy, SciPy, Scikit-learn and Matplotlib official documentation.

The dataset used in this report is a hypothetical sample created for learning and contains no real student information.